今天的内容

- 流水线

今日及接下来几次课程的内容安排

- 单周期微体系结构（Single-cycle Microarchitectures）
- 多周期与微程序化微体系结构（Multi-cycle and Microprogrammed Microarchitectures）
- 流水线技术（Pipelining）
- 流水线中的问题：控制与数据相关性处理、状态维护与恢复等（Issues in Pipelining: Control & Data Dependence Handling, State Maintenance and Recovery, …）

上一讲回顾

- **多周期与微程序化微体系结构（Multi-cycle and Microprogrammed Microarchitectures）**
  - 优势与设计原则（Benefits vs. Design Principles）
  - 何时生成控制信号（When to Generate Control Signals）
  - LC-3b 状态机、数据通路与控制结构（LC-3b State Machine, Datapath, Control Structure）
  - 微程序控制的优点（Advantages of Microprogrammed Control）

回顾：简易的LC-3b控制和数据通路

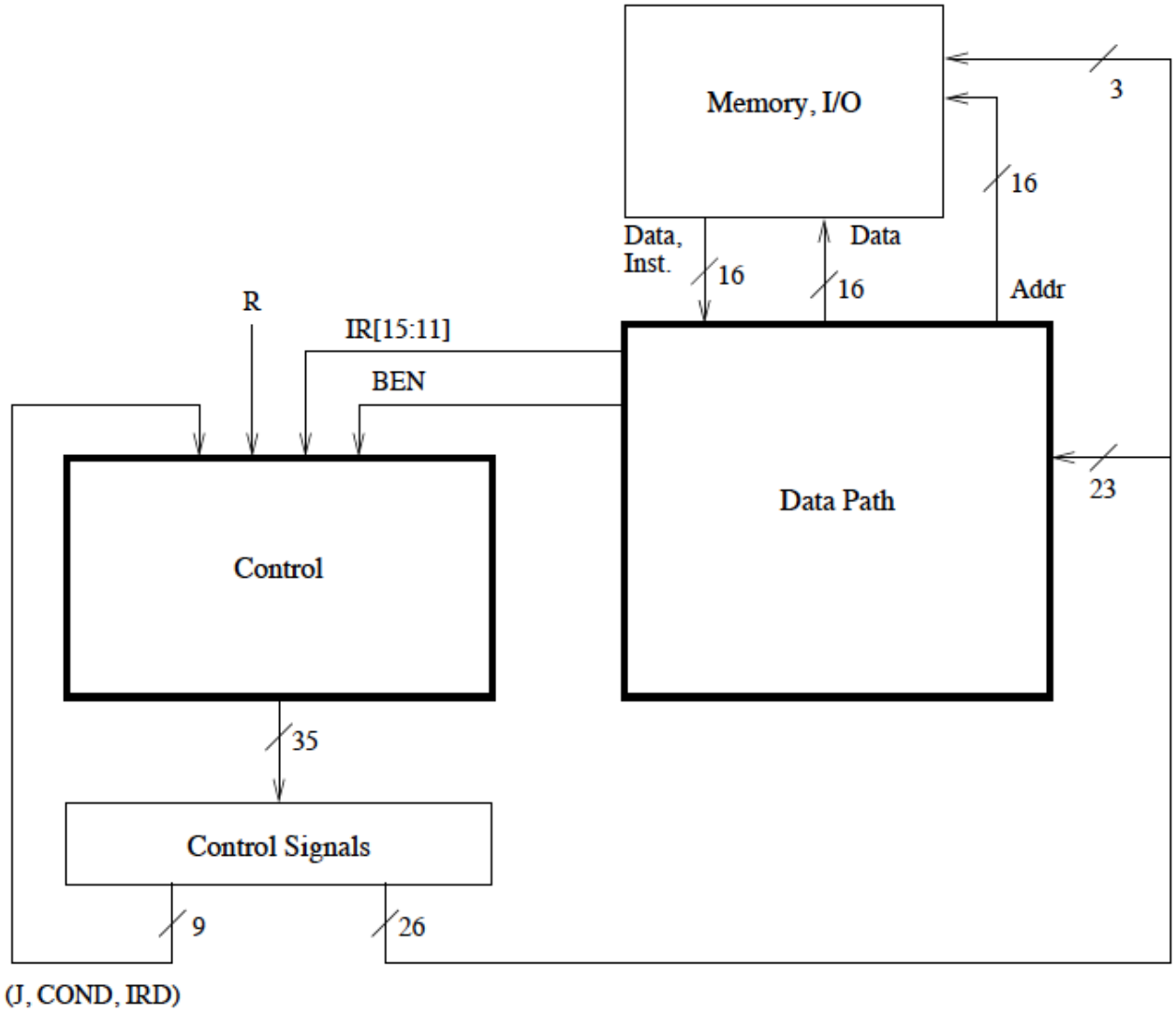
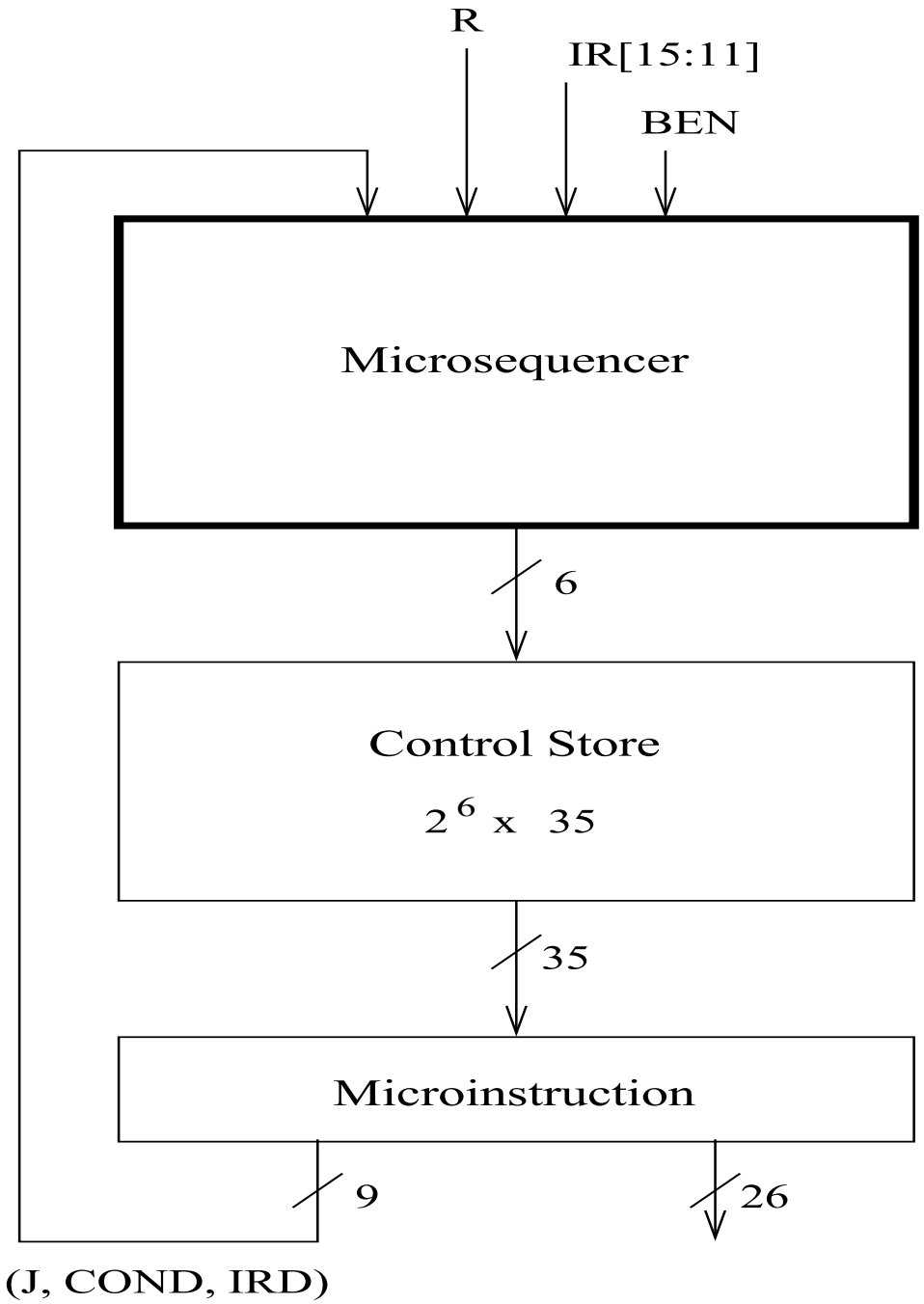


Figure C.1: Microarchitecture of the LC-3b, major components


R
IR[15:11]
BEN
Microsequencer
6
Control Store
2 6 x 35
35
Microinstruction
9
26
(J, COND, IRD)

控制结构的简单设计

回顾：抽象的力量（Power of Abstraction）

- 微指令控制存储（control store of microinstructions） 的概念为硬件设计者提供了一种新的抽象：微程序设计（microprogramming）。
- 设计者可以将任何所需的操作转换为一系列微指令（microinstructions）。
- 设计者所需提供的仅包括：
  - 实现目标操作所需的微指令序列（The sequence of microinstructions）
  - 控制逻辑正确执行这些微指令序列的能力（The ability for the control logic to correctly sequence through the microinstructions）
  - 任何额外需要的数据通路部件和控制信号（Any additional datapath elements and control signals）
    （若该操作可以通过现有控制信号“翻译”实现，则无需额外添加）

回顾：微程序控制的优点

- ✅ **通过控制数据通路（使用微序列器）实现强大计算能力的简化设计**
  - 高级指令集架构（ISA）可被翻译为微代码（microcode，即一系列微指令 u-instructions）
  - 微代码（u-code）使得最小化的数据通路即可模拟一个完整的 ISA
  - 微指令可以被看作一种用户不可见的指令集（u-ISA）
- ✅ **易于扩展指令集架构（ISA）**
  - 只需修改微代码即可支持新的指令
  - 可以通过一系列简单的微指令来支持复杂指令
- ✅ **可更新机器行为**
  - 通过修改微代码即可修复指令实现中的错误（即使在现场环境中）

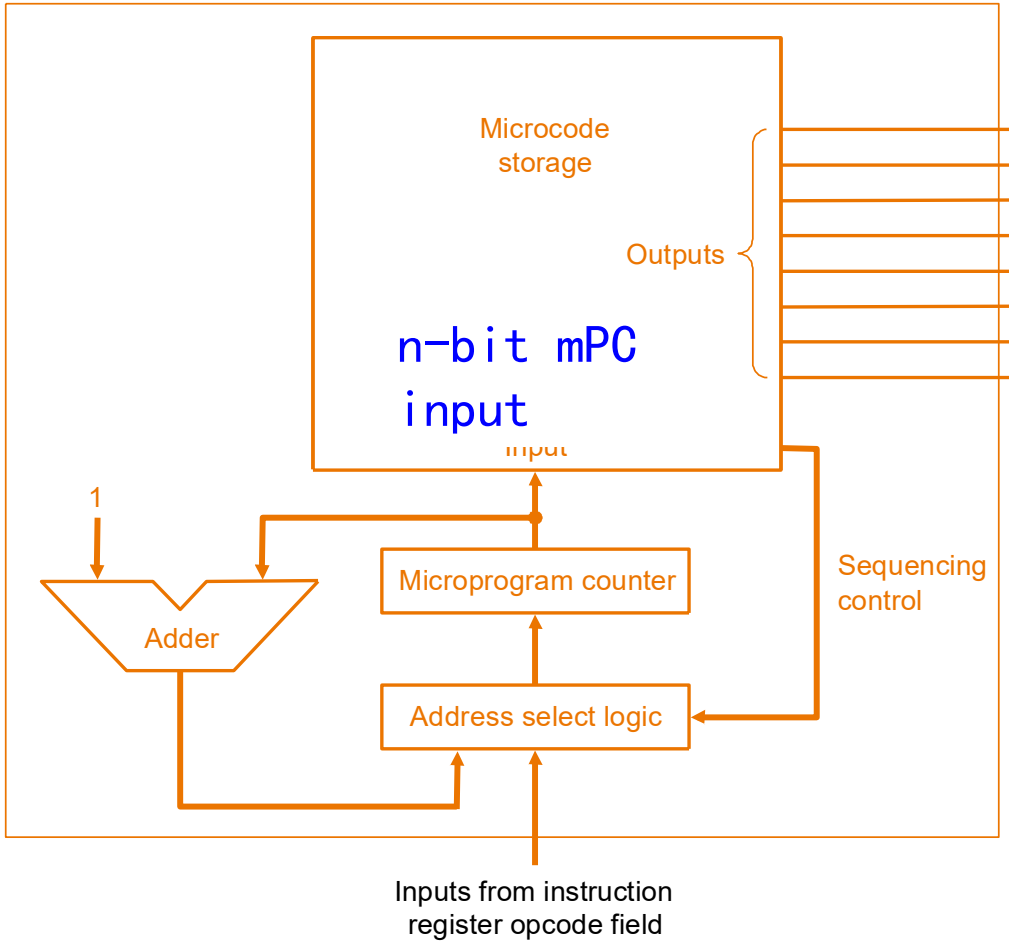
总结：微程序控制

- 水平微代码与垂直微代码（Horizontal vs. Vertical Microcode）
- 纳米码、微码与毫码的比较（Nanocode vs. Microcode vs. Millicode）
- 微程序化的 MIPS：一个示例（Microprogrammed MIPS: An Example）

水平微码（Horizontal Microcode）

- 单一控制存储器（control store）提供所有控制信号

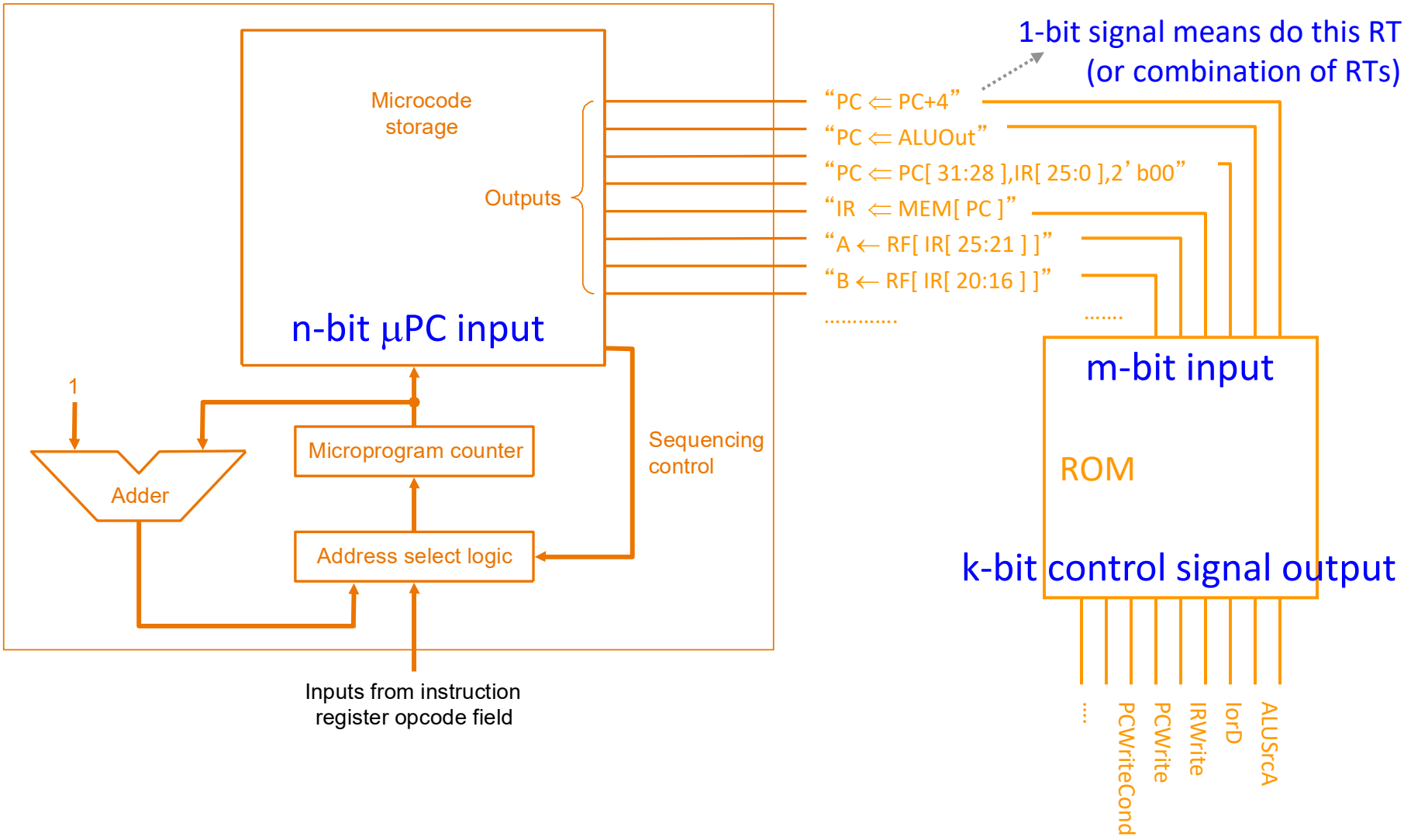
以 MIPS 设计为例



k 位控制信号输出（k-bit control signal output）

垂直微码（Vertical Microcode）

- 两级控制存储结构：第一级指定抽象操作



纳米码（Nanocode）与毫码（Millicode）

- **纳米码（Nanocode）：位于传统微代码之下的一层**
  - 为子系统提供微程序化控制（例如：复杂的浮点运算模块），该子系统作为从属单元运行在一个微控制的数据通路中。
- **毫码（Millicode）：位于传统微代码之上的一层**
  - 属于 ISA 级的子程序，可由微控制器调用，用于处理复杂操作和系统功能。
  - 例如：E.g., Heller and Farrell, “Millicode in an IBM zSeries processor,” IBM JR&D, May/Jul 2004.

- **在这两种情况下，我们都避免让主微控制器（main μ-controller）变得过于复杂。**
- **可以将这些视为在不同抽象层次上的“微代码”（microcode at different levels of abstraction）。**

纳米码（Nanocode）与毫码（Millicode）

所以现在我们有了三个层次：

层级	名称	控制范围	典型功能	抽象层次
Millicode	毫码	系统级（ISA 扩展）	异常处理、复杂操作	高
Microcode	微码	指令级	指令执行控制	中
Nanocode	纳米码	模块级（子系统）	浮点单元、缓存控制	低

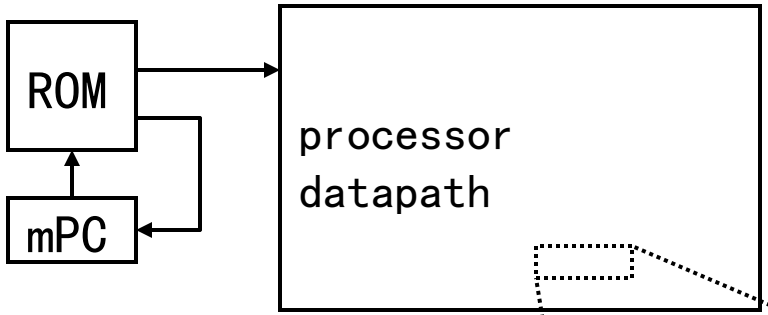
可以看出，**它们的区别不在于形式，而在于抽象层次与作用范围。**

你可以把它们想象成一棵树：

- 主干是微码；
- 根部延伸出纳米码（控制更底层的电路）；
- 枝叶伸展出毫码（处理更高层的任务）。

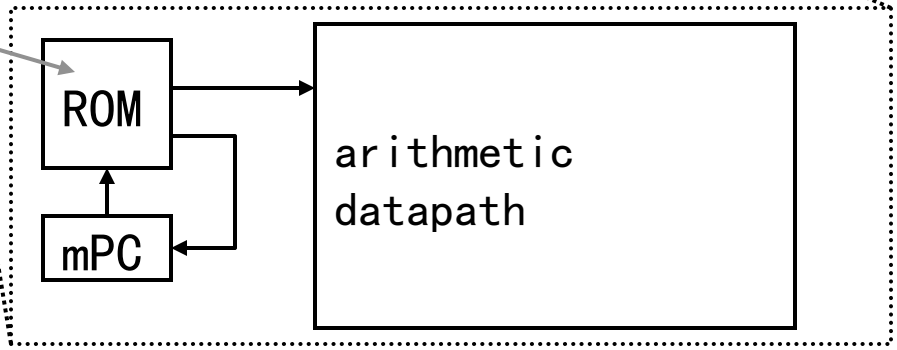
纳米码概念示意图

一个“微代码化（μcoded）”的处理器实现



当一个微代码化子系统被嵌入到另一个微代码化系统中时，我们称之为“纳米码（nanocode）”。

一个“微代码化（μcoded）”的浮点单元（FPU）实现



微代码化的多周期 MIPS 设计（Microcoded Multi-Cycle MIPS Design）

- 任何指令集架构（ISA）都可以通过微程序化微体系结构实现。（Any ISA can be implemented with a microprogrammed microarchitecture）

多周期 vs. 单周期 微体系结构（Multi-Cycle vs. Single-Cycle uArch）

- 优点（Advantages）
- 缺点（Disadvantages）

微程序控制 vs. 硬连线控制（Microprogrammed vs. Hardwired Control）

- 优点（Advantages）

- 缺点（Disadvantages）

我们能做得更好吗？

- 问题： 你认为多周期设计有哪些局限？（What limitations do you see with the multi-cycle design?）

- 并行度受限（Limited concurrency）
  - 一些硬件资源在指令处理的不同阶段处于空闲状态。
  - 当一条指令正在“译码（decode）”或“执行（execute）”时，**取指（fetch）逻辑**处于空闲。
  - 当进行内存访问（memory access）时，大部分数据通路都处于空闲状态。

我们能否利用空闲硬件来提升并行度？

- 目标（Goal）：
  更多的并行执行 ➜ 更高的指令吞吐率（即在一个周期中完成更多“工作”）。
- 思路（Idea）：
  当一条指令在其执行阶段使用某些硬件资源时，可以让其他指令利用当前未被占用的硬件资源进行处理。
- 例如：
  - 当一条指令正在译码时，可以**取下一条指令**。
  - 当一条指令正在执行时，可以**译码另一条指令**。
  - 当一条指令正在访问数据内存（load/store）时，可以**执行下一条指令**。
  - 当一条指令正在将结果写回寄存器文件时，可以**为下一条指令访问数据内存**。

今天的内容

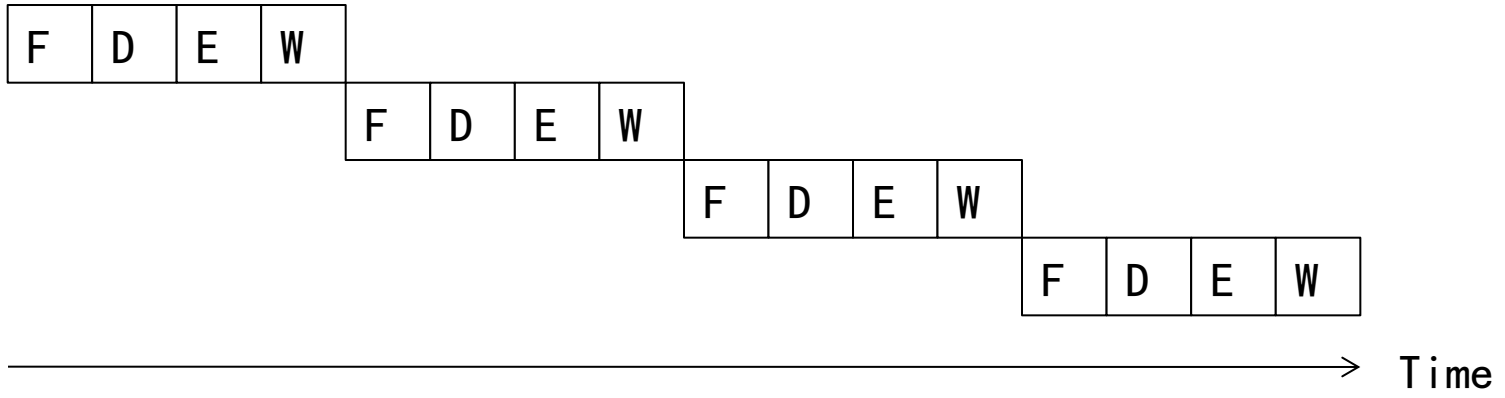
- 流水线

流水线：基本思想（Pipelining: Basic Idea）

- **更系统地说（More systematically）：**
  - 将多条指令的执行过程进行**流水化（pipeline）**。
  - 类比：指令的“装配线式处理（Assembly line processing）”。
- **核心思想（Idea）：**
  - **将指令处理周期划分为若干独立的“阶段”（stages）。**
  - 确保每个阶段拥有足够的硬件资源，以便在每个阶段能同时处理一条指令。
  - **在每个阶段处理不同的指令。**
    - 程序顺序上相邻的指令在流水线中被处理于连续的阶段。
- **优点（Benefit）：**
  - 提高指令处理吞吐率（instruction throughput），即增加每周期执行指令数（1/CPI）。
- **缺点（Downside）：**
  - 思考流水线可能带来的问题……

示例：执行四条相互独立的加法指令

- 多周期（Multi-cycle）：每条指令需要 4 个周期

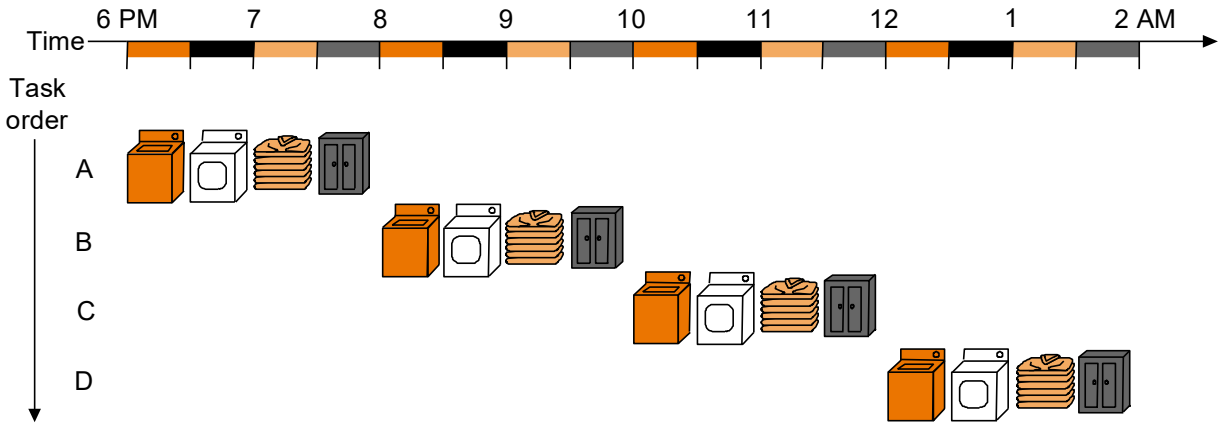


- 流水线（Pipelined）：4 个周期执行 4 条指令（稳定状态下）



生活总是这么美好吗？

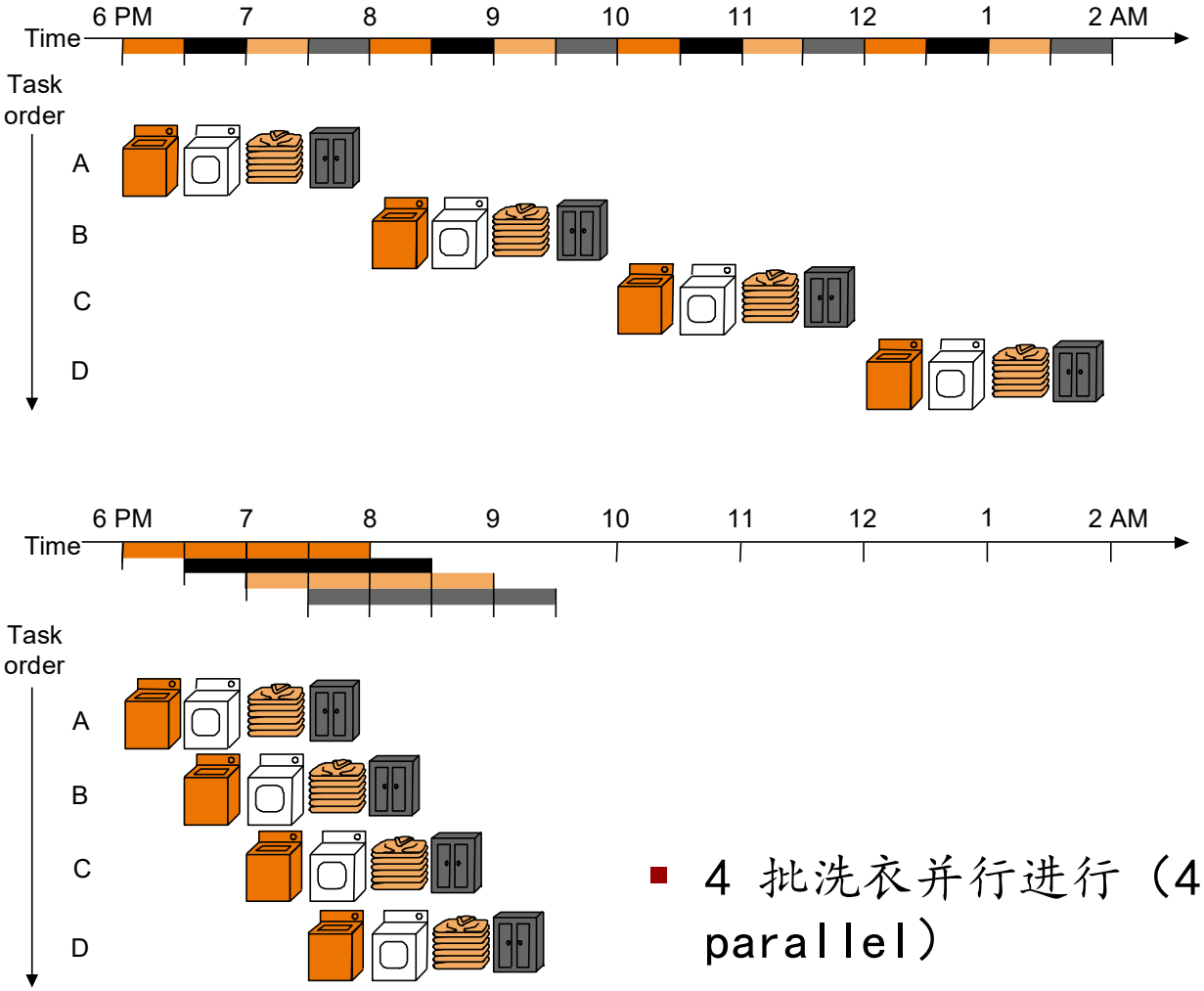
洗衣类比（The Laundry Analogy）



1. “把一筒脏衣服放进洗衣机”
2. “洗衣机完成后，把湿衣服放入烘干机”
3. “烘干机完成后，取出干衣服并折叠”
4. “折叠完成后，收衣服”

- 一次洗衣的各步骤是顺序依赖的（sequentially dependent）。
- 不同批次之间没有依赖关系（no dependence between different loads）。
- 不同步骤不共享资源（do not share resources）。

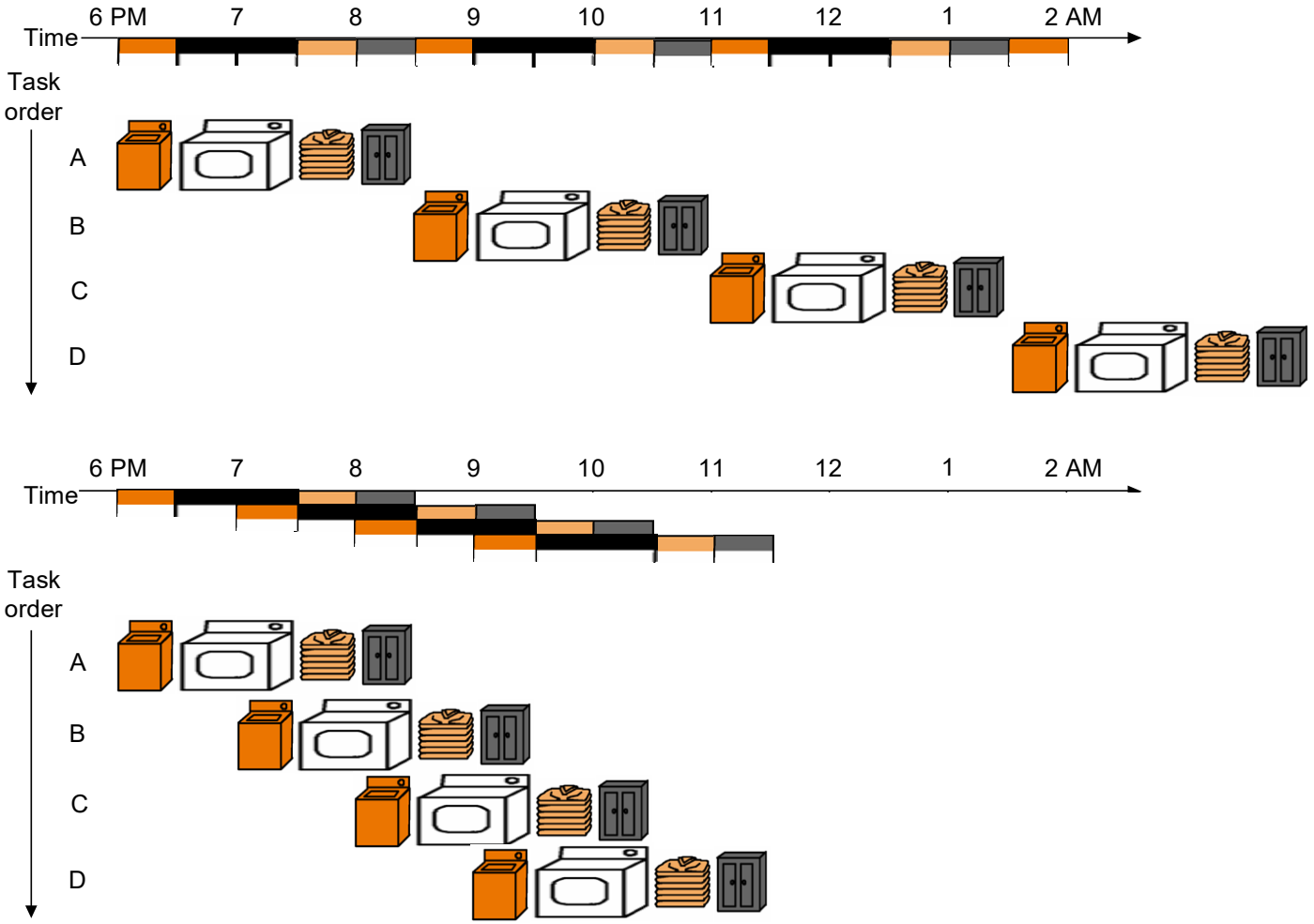
多批洗衣的流水化处理



- 4 批洗衣并行进行（4 loads of laundry in parallel）
- 无需额外资源（no additional resources）
- 吞吐率提高了 4 倍（throughput increased by 4）

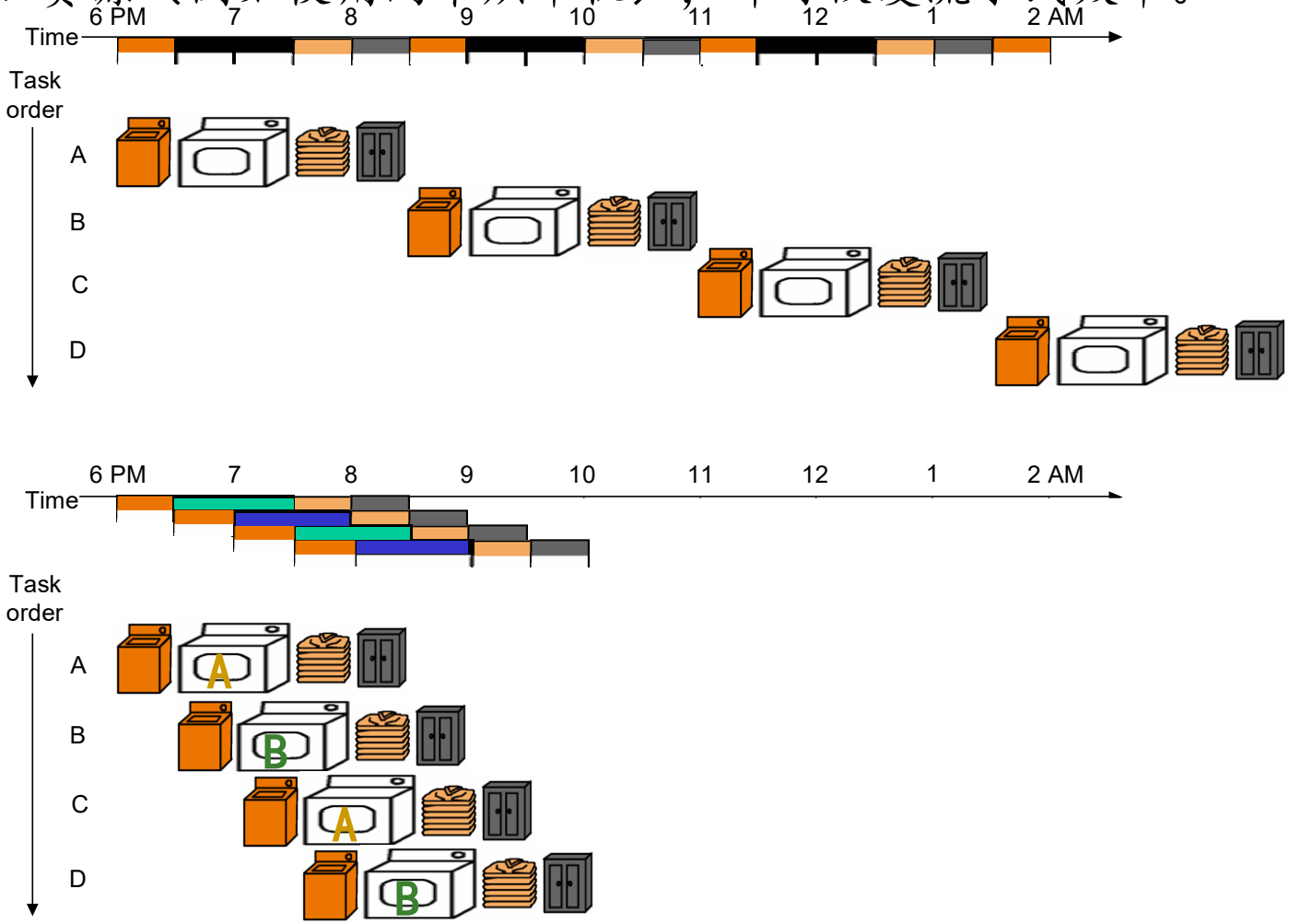
多批洗衣的流水化：实际情况

- 实际上：当洗衣、烘干、折叠时间不一致时，流水线的速度由最慢的阶段决定（the slowest step decides throughput）。



多批洗衣的流水化：实践优化

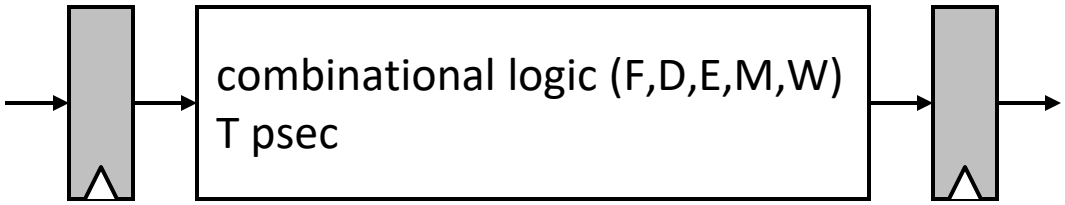
- 解决办法：
  - 增加资源（例如使用两个烘干机），即可恢复流水线效率。



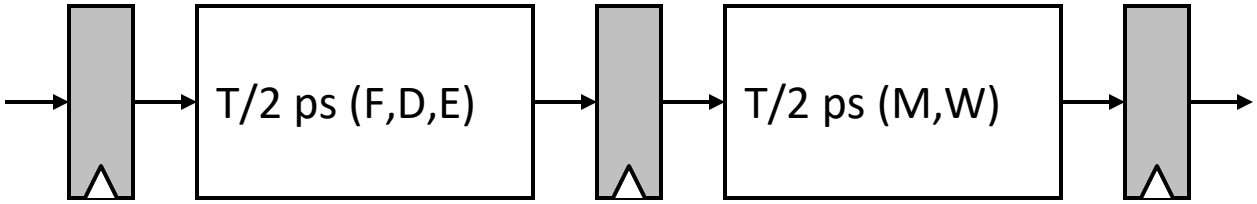
理想的流水线（An Ideal Pipeline）

- **目标（Goal）：以**极少的成本增加**来提升吞吐率。（在指令处理的情况下，成本指硬件开销。）**
- **重复执行相同操作**
  - 同一个操作在大量不同输入上重复执行。（例如：所有洗衣任务都经历相同的步骤。）
- **重复执行相互独立的操作**
  - 不同操作之间不存在依赖关系。
- **可均匀划分的子操作**
  - 处理过程可以被均匀地划分为多个具有相同延迟的子阶段，且这些阶段**不共享资源**。
- **典型示例：**
  - 汽车装配线、洗衣任务。
  - 那么，对于**指令处理周期（instruction processing “cycle”）**，是否也可以这样实现呢？

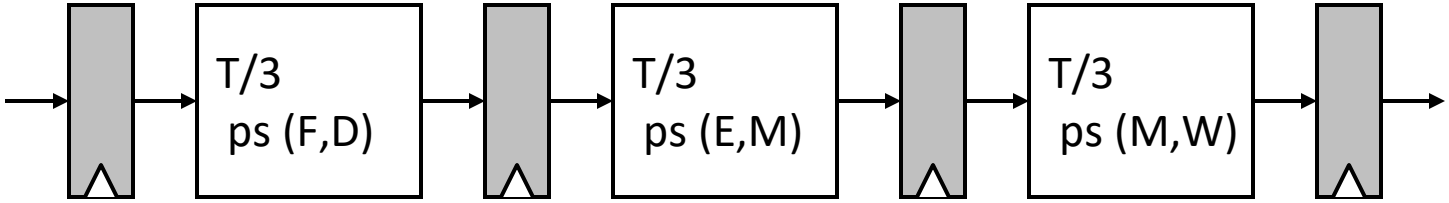
理想化的流水线



BW=~(1/T)



BW=~(2/T)



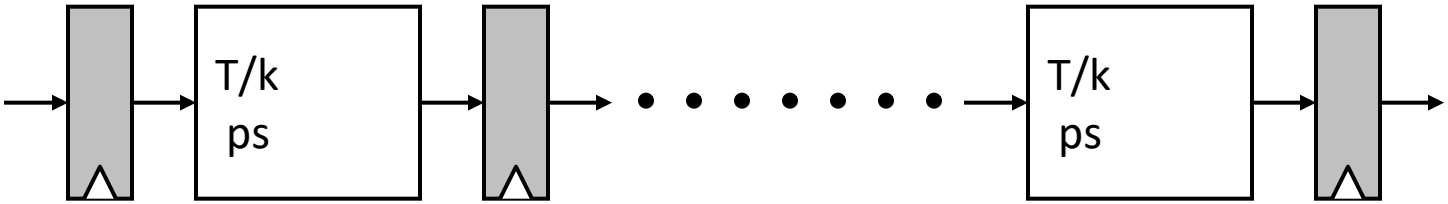
BW=~(3/T)

更现实的流水线：吞吐率

- 非流水线版本（Nonpipelined version）延迟为 T
  - $BW = 1/(T + S)$
  - 其中 S = 锁存器延迟（latch delay）。



- k 级流水线版本（k-stage pipelined version）
  - $BW_{k-stage} = 1/(T/k + S)$
  - $BW_{max} = 1/(1\ gate\ delay + S)$
  - 锁存器延迟会降低吞吐率
    （阶段之间的切换开销会带来性能损失）

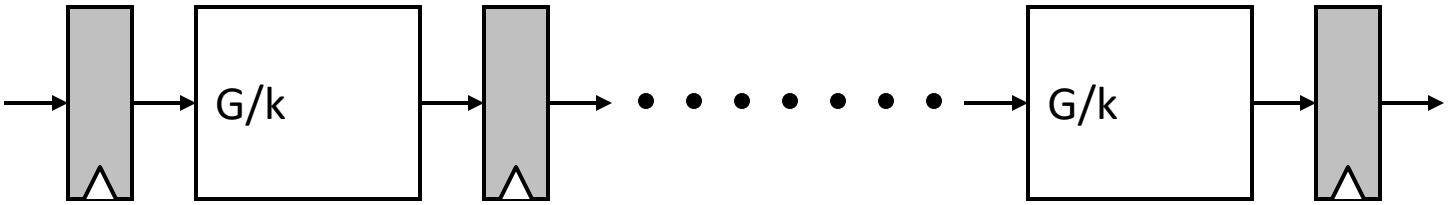


更现实的流水线：成本

- 非流水线版本（Nonpipelined version）组合逻辑成本为 G
  - Cost = $G + L$ 其中 L = 锁存器成本（latch cost）。



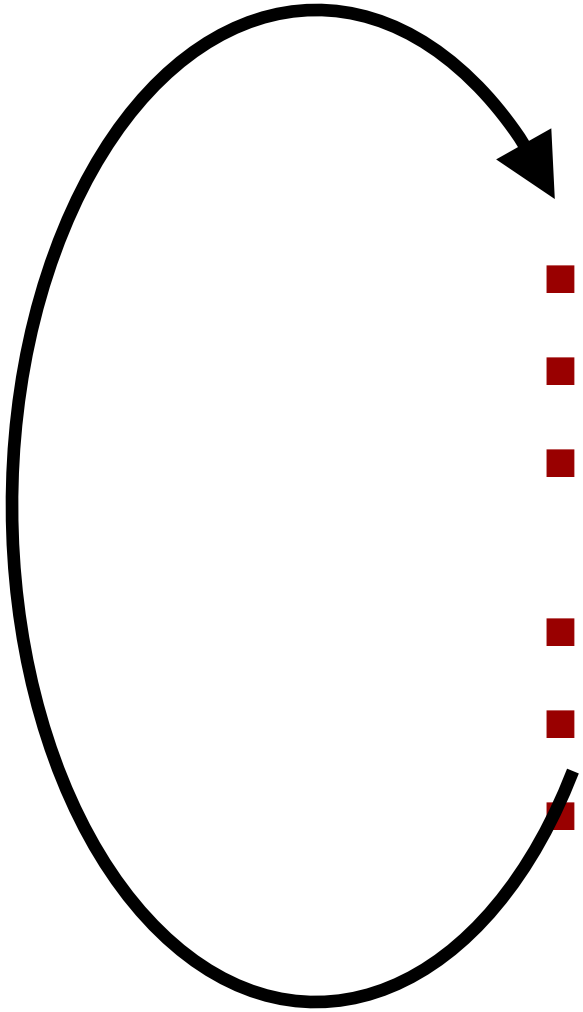
- k 级流水线版本（k-stage pipelined version）
  - $\text{Cost}_{k-stage} = G + Lk$
  - 锁存器的数量随流水线级数增加，从而提高硬件成本。



今天的内容

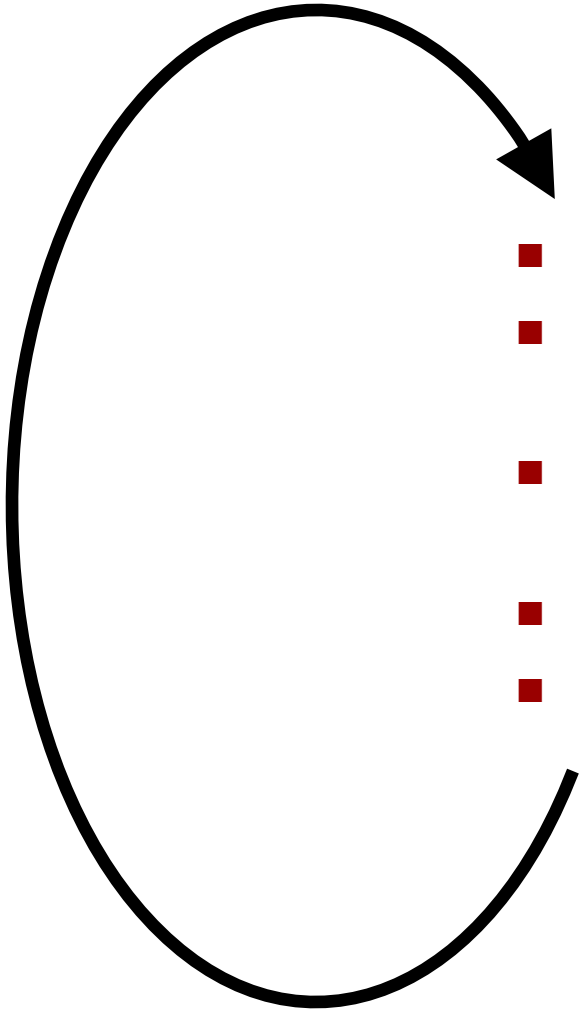
- 流水线指令处理

回顾：指令处理周期



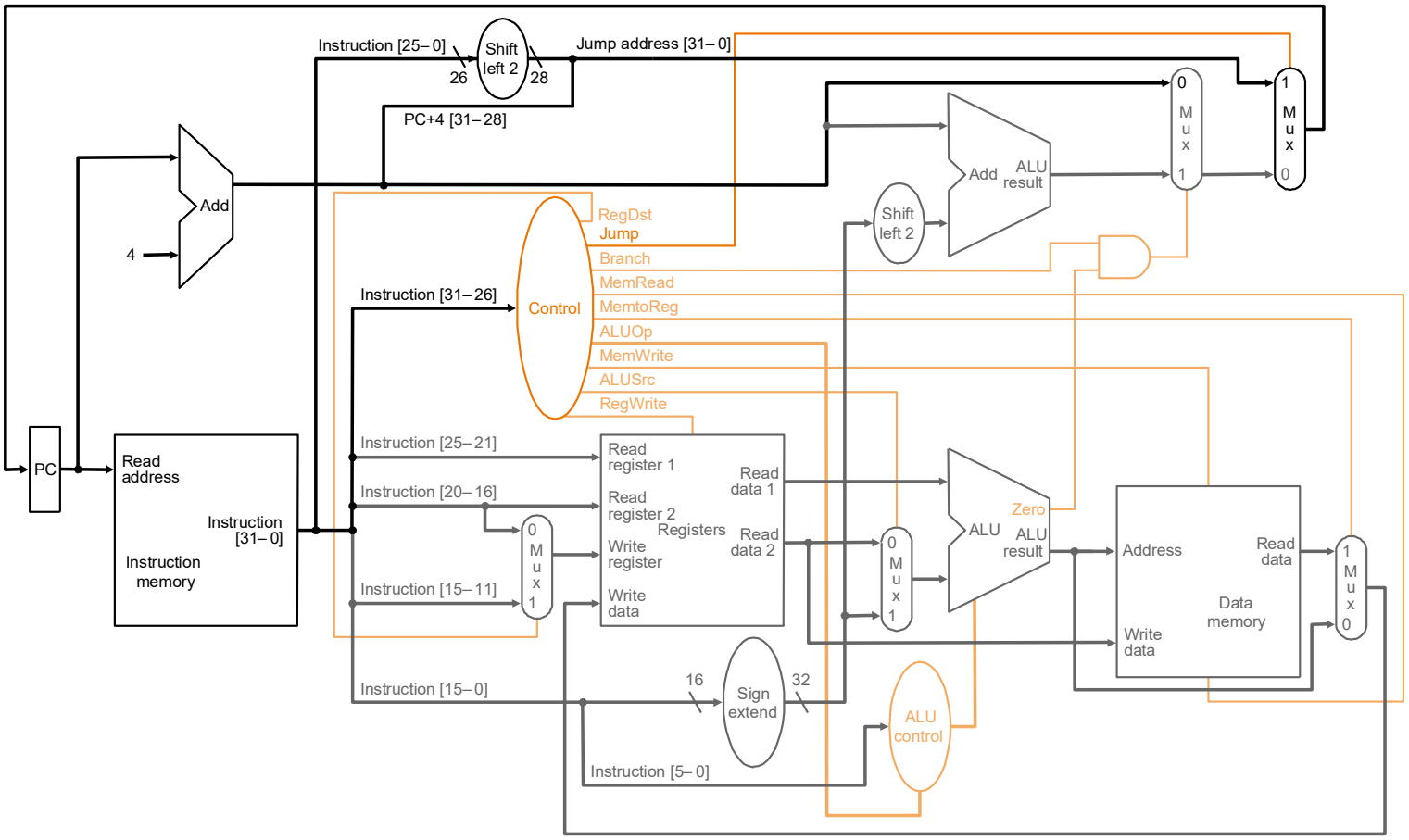
- 取指（Fetch）
- 译码（Decode）
- 计算地址（Evaluate Address）
- 取操作数（Fetch Operands）
- 执行（Execute）
- 存储结果（Store Result）

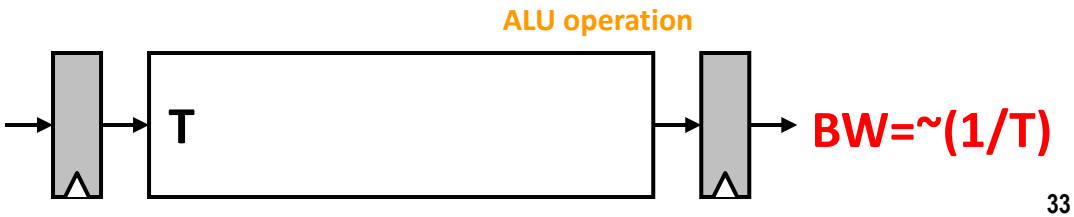
回顾：指令处理周期



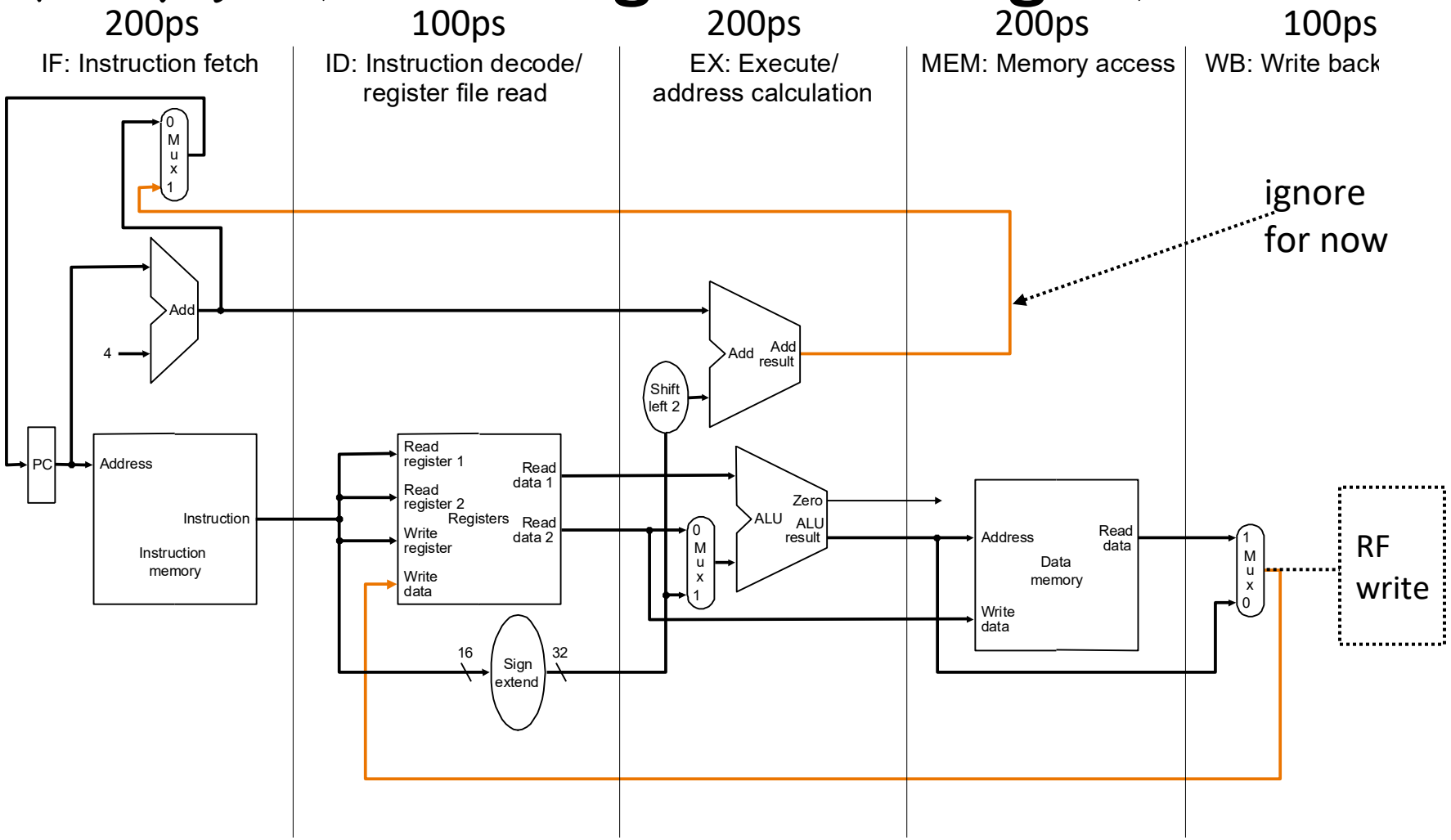
- 指令取指（Instruction Fetch, IF）
- 指令译码与寄存器操作数取值（Instruction Decode and Register Operand Fetch, ID/RF）
- 执行 / 计算内存地址（Execute / Evaluate Memory Address, EX/AG）
- 内存操作数访问（Memory Operand Fetch, MEM）
- 结果写回（Store / Writeback Result, WB）

回顾单周期微架构（Remember the Single-Cycle μArch）



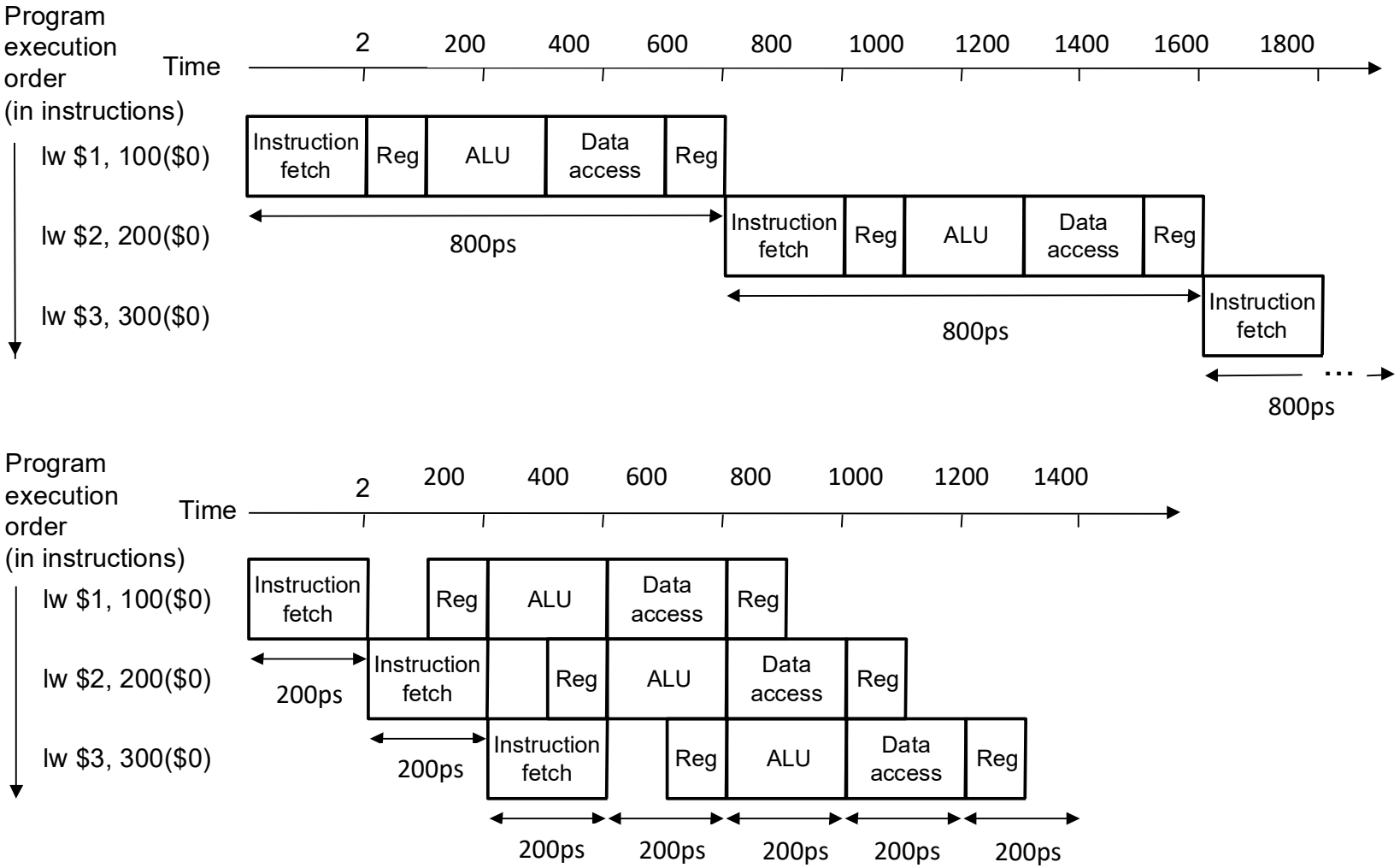


阶段划分（Dividing Into Stages）



- 这是正确的划分方式吗？
- 为什么不是 4 个或 6 个阶段？
- 为什么不采用不同的分界点？

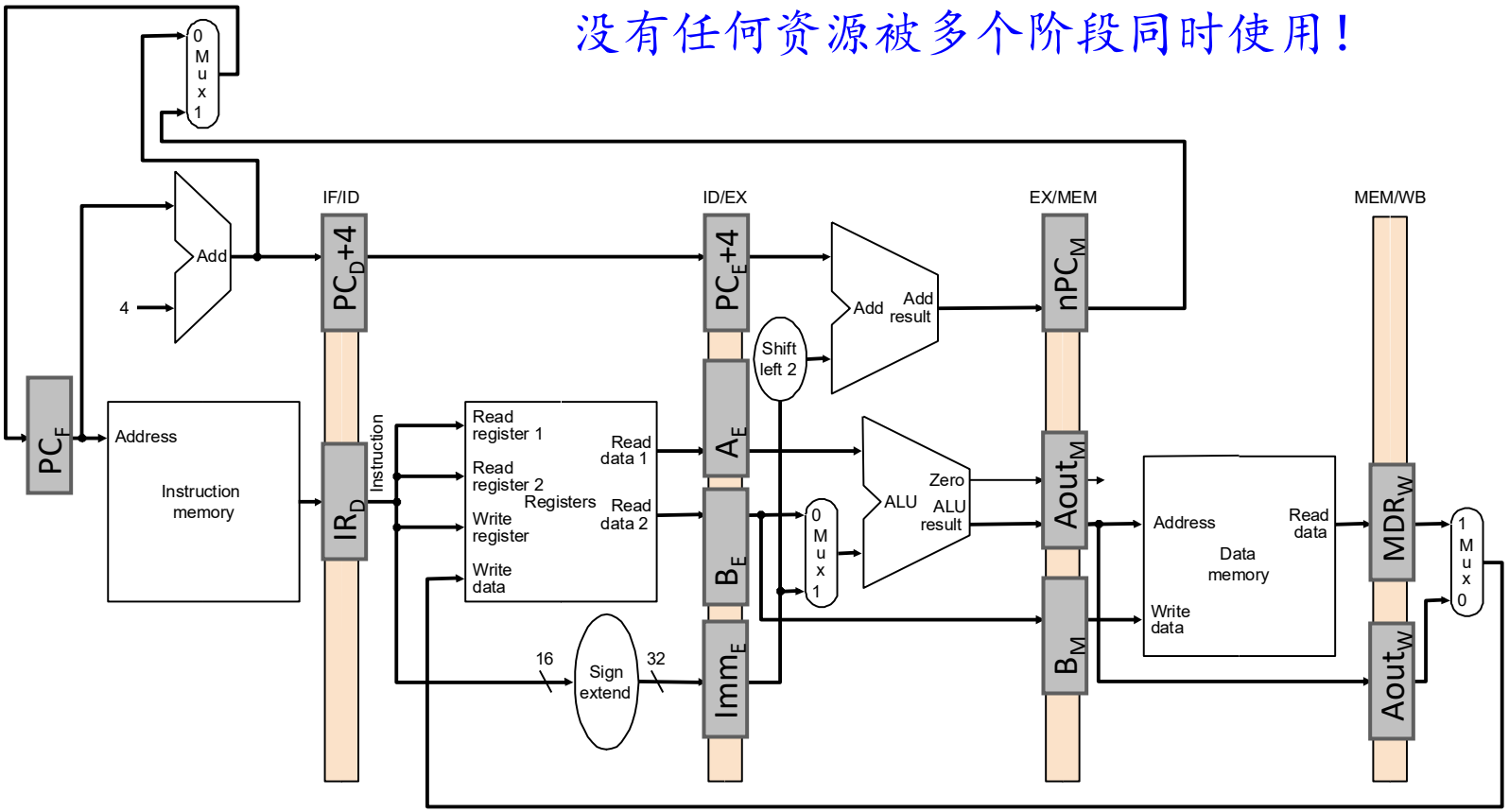
指令流水线的吞吐率

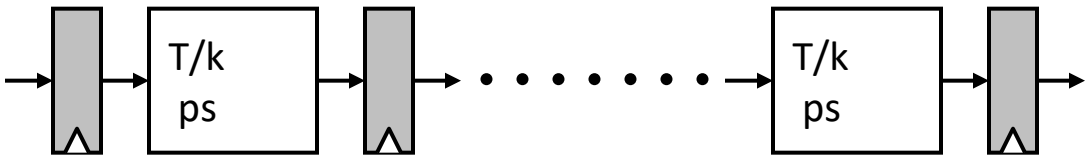


- 5 级流水线的加速比是 4，而不是理想模型预测的 5。为什么？

实现流水线处理：流水线寄存器

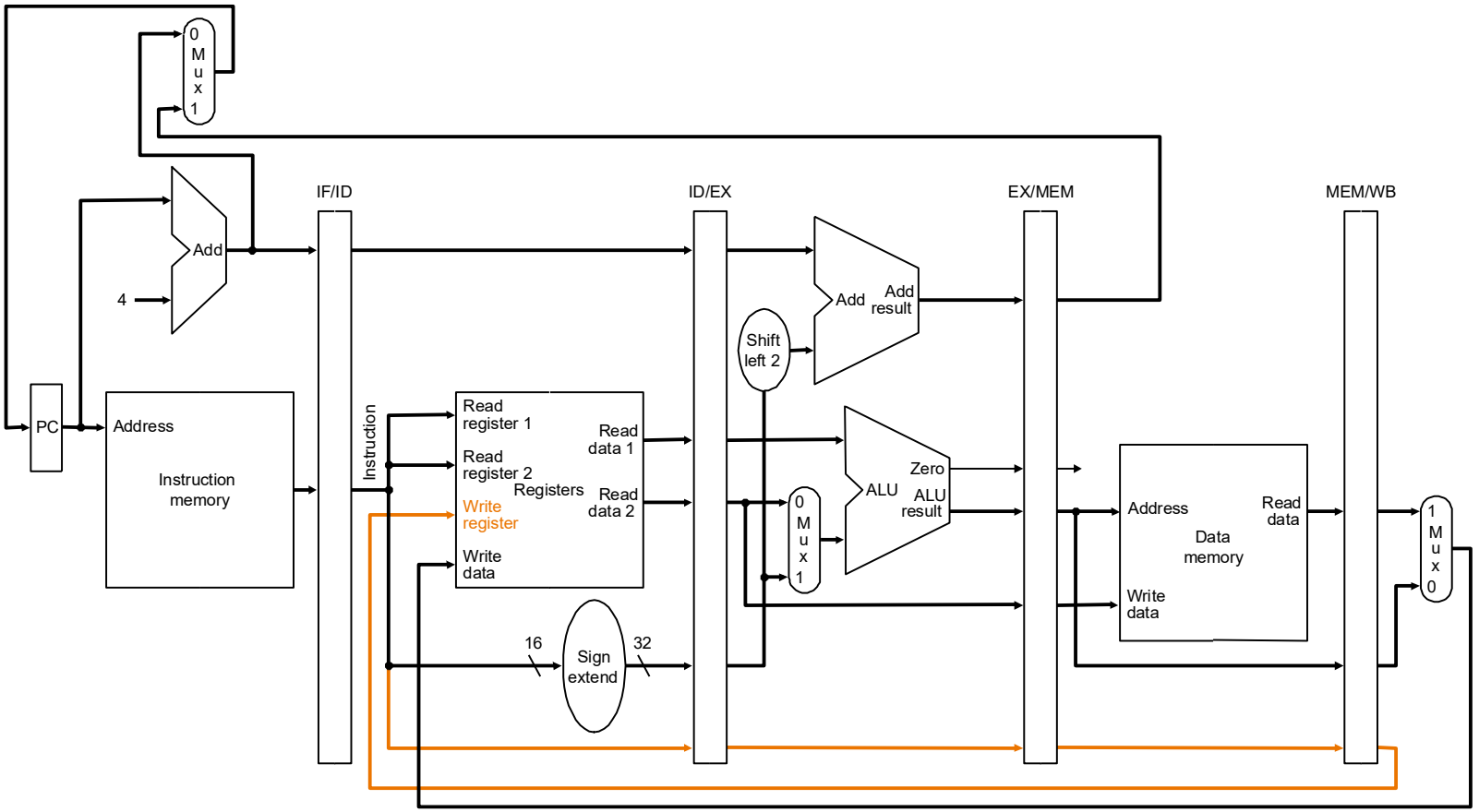
没有任何资源被多个阶段同时使用！



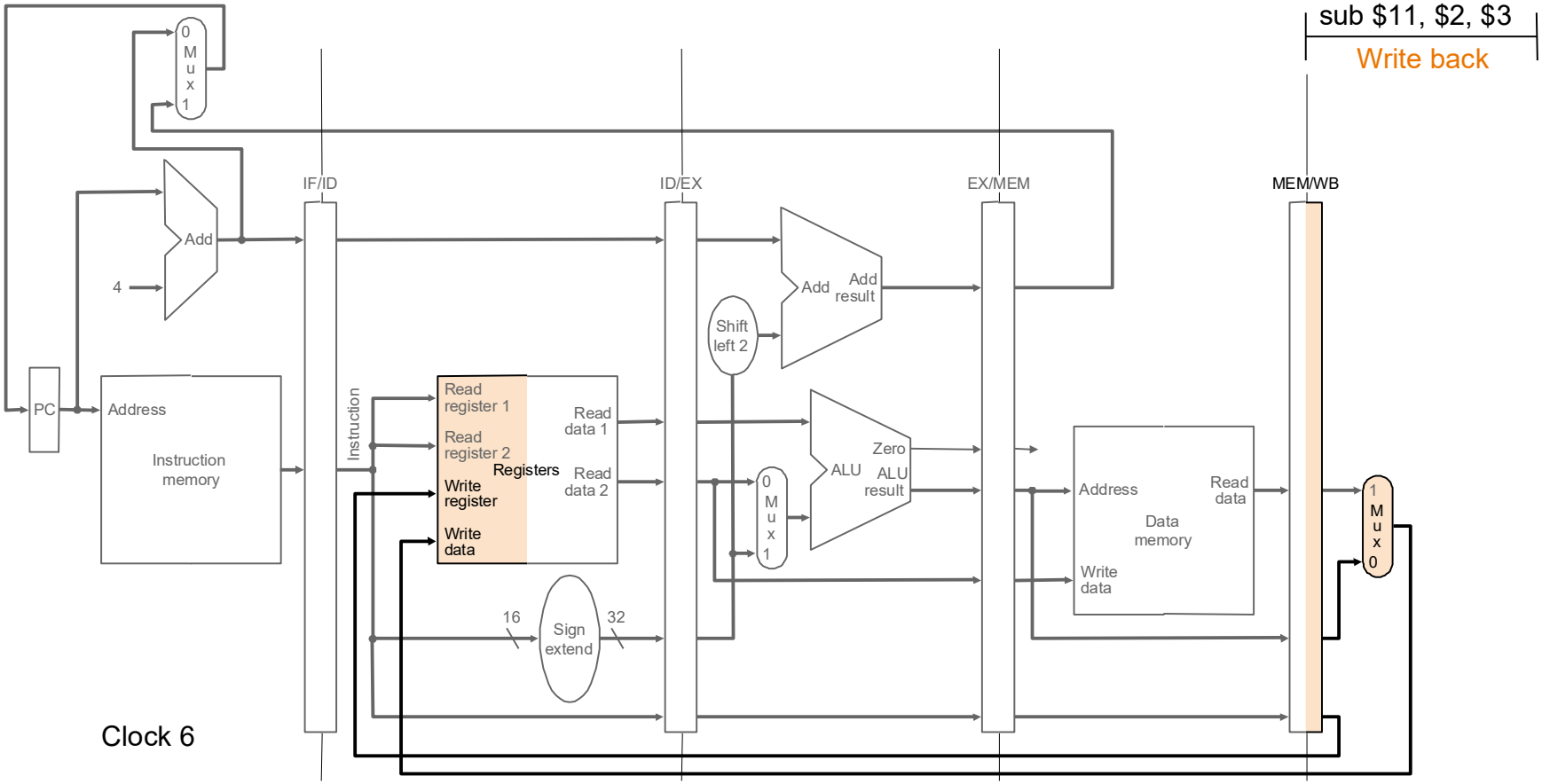


流水线操作示例

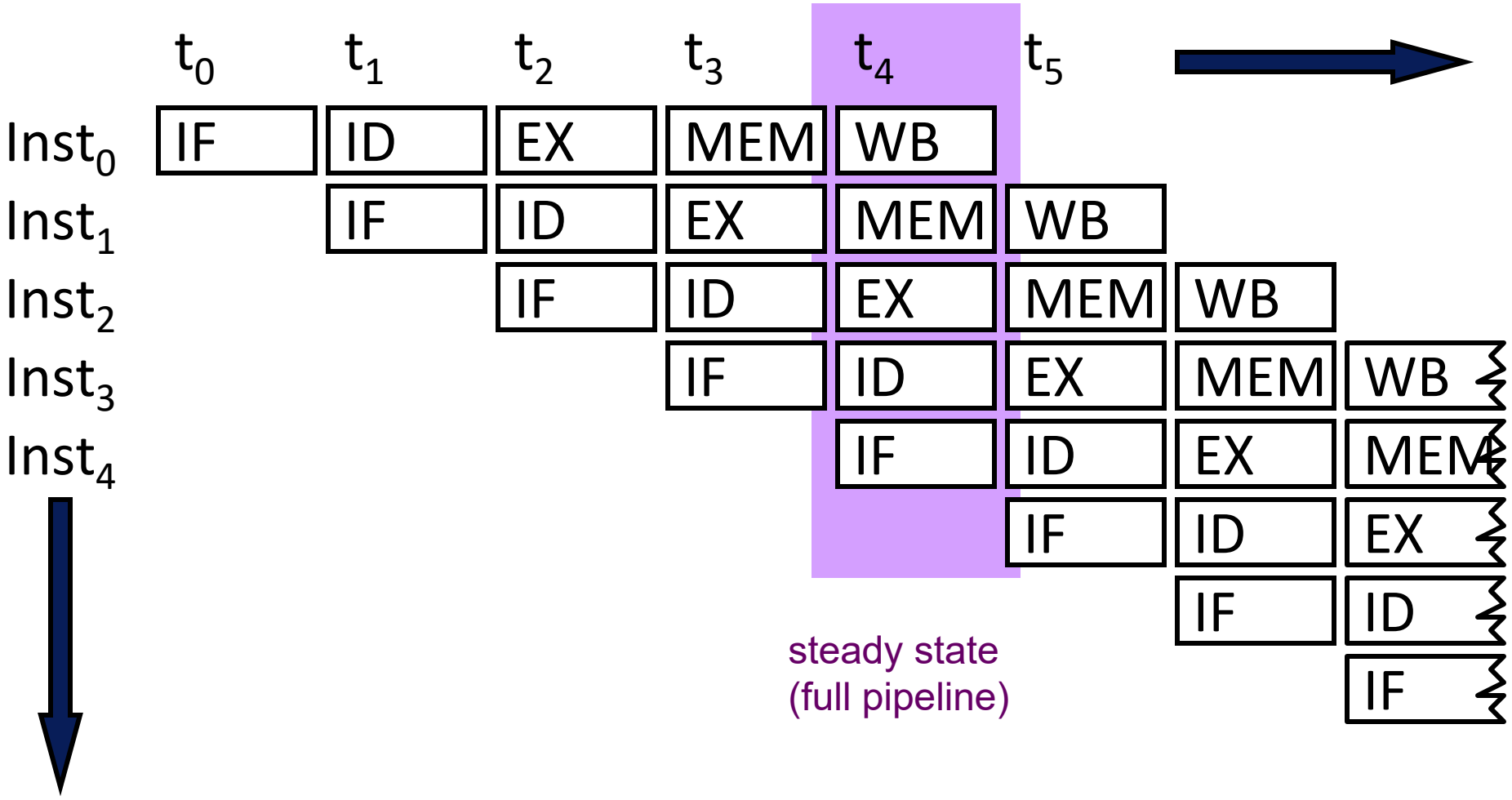
- 所有类型的指令都必须遵循相同的路径与时序，经过流水线的各个阶段。这会对性能产生影响吗？



流水线操作示例


sub $11, $2, $3
Write back
IF/ID
ID/EX
EX/MEM
MEM/WB
0
M
u
x
1
Add
4
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Data memory
Read data
Write data
16
Sign extend
32
1
M
u
x
0
Clock 6

流水线操作示意：操作视图



流水线操作示意：资源视图

	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
IF	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	I_{10}
ID		I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9
EX			I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8
MEM				I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
WB					I_0	I_1	I_2	I_3	I_4	I_5	I_6